ANTHROPIC RUNTIME CONFIGURATION

ARC

A complete **development operating system** for Claude Code — setup, planning, execution, quality, and retrospectives in one coherent framework.

11 Skills

23 Domain Modules

5 Framework Layers

ARCHITECTURE

Five integrated layers, no re-explaining between sessions

CONTEXT

Domain expertise, project brief, CLAUDE.md, voice calibration. Every session starts fully loaded.

`/arc-init`
`/arc-kickoff`
`/arc-resume`
`/arc-export`

PLANNING

Architectural rigor before a line of code. Maps failure modes, delight opportunities, reversibility.

`/arc-plan`
EXPAND / HOLD / REDUCE

EXECUTION

Pre-PR code review, fully automated ship pipeline, changelog and PR creation.

`/arc-review`
`/arc-ship`

QUALITY

Anti-slop enforcement, stub detection, deployment safety, wiring verification. Domain-specific checks.

`/arc-check`
`/arc-rekickoff`

MEMORY

SESSION STATE.md keeps every session continuous. Lessons accumulate. Retros extract the signal.

`/arc-retro`
`/arc-progress`
tasks/lessons.md

THE WORKFLOW

Four standard loops covering the full development cycle

NEW PROJECT

/arc-init — 3 questions, CLAUDE.md + tasks/
→ **/arc-kickoff** — 15-min interview, brief + arch diagram
→ **/arc-plan** — EXPAND mode, failure map + roadmap
→ **Build** — Informed by plan, STATE.md tracking

EVERY SESSION

/arc-resume — Loads STATE.md, git + PR signals
→ **Work** — Full context already loaded — domain, constraints, voice, history
→ **/arc-check** — After any significant output

BEFORE SHIP

/arc-check — Stubs, security, wiring, deploy safety
→ **/arc-review** — Two-pass diff, CRITICAL → INFO
→ **/arc-ship** — Tests → version → changelog → commits → push → PR (fully automated)

SPRINT CLOSE

/arc-retro — What shipped, what broke, what to change
→ **/arc-progress** — Context health, open questions count
→ **lessons.md** ↑ — Lessons append — never overwrite. Institutional knowledge accumulates across sprints.

SKILLS REFERENCE

11 skills across three groups

SETUP & CONTEXT

/arc-init New project. Creates CLAUDE.md + tasks/ in 2 min.

/arc-kickoff Deep context: 15-min interview, brief, arch diagram, export.

/arc-resume Start of every session. Reads STATE.md, surfaces blockers + PRs.

/arc-rekickoff Mid-project pivot or scope change. Re-orients context.

/arc-export Portable context prompt. Paste into Claude.ai or share.

PLANNING & EXECUTION

/arc-plan Before writing code. EXPAND (default): finds the real product, maps delight + failure modes. Also: HOLD / REDUCE.

/arc-review Pre-PR. Two-pass diff: CRITICAL (security, auth, IDOR) → INFORMATIONAL (quality, tests).

/arc-ship Automated pipeline: tests → review → version → changelog → commits → push → PR.

QUALITY & RETRO

/arc-check After any significant output. Stubs, wiring, specificity, deploy safety.

/arc-progress Project health dashboard: context health, open questions, todo summary.

/arc-retro End of sprint. Reviews what shipped, what broke. Appends to lessons.md.

COMPOSITION MODEL

```
base.md + overlay + [domain] + overrides
→ your project's CLAUDE.md
```

Fintech SaaS	base + software-eng + fintech + saas
YouTube Channel	base + content-publishing + education
Market Analysis	base + business-strategy + saas
Learning Rust	base + learning-research + developer-tools

Target: under 3,000 words · /arc-init composes automatically

THREE NON-NEGOTIABLES

1. **Plan before building**
   Non-trivial tasks start with a written plan in `tasks/plan.md`. Prevents the most common Claude failure: implementing before understanding.
2. **Anti-slop enforced**
   Every output must contain project-specific information. If it could apply to any project in the same category, it's generic and gets rejected.
3. **Evidence before done**
   Code needs clean diagnostics. Builds need exit code 0. Plans need verification steps. Nothing is marked complete without proof.

```
$ npx arc-setup
```

Setup, plan, build, review, ship, retro — one coherent loop.

MIT · github.com/[your-repo]/arc